Exercise Set 5

13 October 2025

1 Wrapping up Hoare Logic

1.1 Binary Search

```
binarysearch(a, elem) =
  l := 0;
  u := a.length-1;
  while (l <= u) do
    m := (u-l)/2;
    if (a[m] < elem) then
      l := m+1
    else
      if (a[m]>elem) then
        u := m-1
      else
        m;
      endif;
    endif
  done;
  -1
```

- Implement it on why3
- As a precondition, we want the array to be sorted in input
- As for the postcondition, the index of the element `elem` should be in the valid range of `a` indices and it should match the element `elem`
- Guess the invariants!

1.2 Bubblesort

Here a pseudocode of the bubblesort algorithm

```
bubblesort(a) =
  i := a.length-1;
  while (i >= 1) do
    j := 0;
```

```
    while (j <= i-1) do
      if (a[j+1] < a[j]) then
        aux := a[j+1];
        a[j+1] := a[j];
        a[j] := aux
      endif;
      j := j+1
    done;
    i := i-1
  done;
  a
```

Proceed step by step with the implementation on why3

- Implement it on why3
- What should be the postconditions? How the input array and the output sorted array are related? Look at the following function permut_all
- We have two nested while loops, hence we need to define invariants for every while loop
- Hint: you can define some separate helper functions (they will be useful for the next and will reduce verbosity). You can define them as `predicate` on why3.

1.3 Selection Sort

Here a pseudocode of the selectionsort algorithm

```
seelctionsort(a) =
  i := 0;
  while (i <= a.length-2) do
    min := i;
    j := i+1;
    while (j <= a.length-1) do
      if (a[j] < a[min]) then
        min := j
      endif;
      j := j+1
    done;
    aux := a[min];
    a[min] := a[i];
    a[i] := aux;
    i := i+1
  done;
  a
```

Proceed step by step with the implementation on why3

- Implement it on why3
- What should be the postconditions? How the input array and the output sorted array are related? Look at the following function permut_all
- We have two nested while loops, hence we need to define invariants for every while loop
- Hint: you can define some separate helper functions (they will be useful for the next and will reduce verbosity). You can define them as `predicate` on why3.